第 12 章練習解答　檔案系統實作

題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS12-E01　Indexed allocation

1. 每個 pointer block 有 4096/4=1024 entries，每 entry 指 4 KiB data，資料容量 4 MiB；再加 12×4 KiB direct。
2. double indirect 第一層有 1024 pointers，每個指向 1024-entry block，共 1,048,576 data pointers。
3. 忽略 inode pointer 本身，每個 indirect block 4 KiB，3 blocks 共 12 KiB metadata；其中還有可能跨多層。

檢查。 每個乘法都要說明單位。

OS12-E02　Crash consistency

1. 可能留下沒有名稱的 orphan inode/block。恢復可以回收它，但無法讓 create 變成成功。
2. 在題設 redo-only/no-steal 模型中，完整 after-images 先持久、commit 記錄再持久；未提交的 home metadata 不得先行修改。恢復時丟棄未提交交易，對已提交交易重放完整 inode 與 directory after-images；其他允許未提交更新寫入 home 的 journal 可能需要 undo。
3. 不能。read 成功若只命中 volatile cache，沒有 durable write/commit，斷電後仍可能不存在。

檢查。 區分 atomic metadata transition 與資料持久性。

OS12-E03　Free block 結構

1. block8的bit0表示allocated；邏輯只查1bit，cache未命中時磁碟仍以儲存block傳輸，不能稱1-bit I/O。
2. 只取一個free block時可由已快取head直接移除，metadata操作可O(1)；若要找連續多塊或重建所有free blocks，才可能遍歷長鏈並增加I/O，兩種操作不能混算。
3. 並行claim需鎖或原子位元操作；持久bitmap、inode與size更新另需journal/COW協定，memory lock本身不提供斷電原子性。

檢查。 不要把 bitmap 查詢成本誤寫成 O(files)。

OS12-E04　Path walk、open table 與 read

1. B4 root cache hit得到home→I3；讀B3 inode table得到I3的directory block B5；讀B5得到u→I5；讀B6得到notes.txt→I7。cold metadata reads=B3/B5/B6，資料I/O另計。
2. offset0涵蓋logical blocks0與1，讀B8/B9；P1 offset變8192，P2仍0。B10保留給之後同一file read。
3. P1 fd4→O1，O1保存offset8192與status/access flags；P2 fd9→O2，O2保存offset0。O1/O2都引用inode7，inode保存size12288與blocks8–10並關聯共享page cache。獨立open有兩個description，不可用一個S1物件同時聲稱offset獨立。
4. 不變。offset屬P2的open-file description；P1保有自己的description與offset。若兩者經dup或fork共用同一description，才會共享offset，但inode/data仍不等於offset。

檢查。 path lookup、inode cache、system table、per-handle state必須逐步分開。

OS12-E05　Bitmap free space 與 file extent

1. bit=1的位置為7、11、12、13、14、15；三塊連續first-fit為11–13，block7後面的block8–10已配置。
2. blocks11–13由free變allocated，bitmap變0000000100000011，free blocks={7,14,15}；inode7 size=24 KiB、pointers新增11–13。
3. 可能出現inode指向blocks11–13，但bitmap仍說free或被另一allocation取走，造成重複配置。需要journal/COW把bitmap、inode pointer、size與data ordering組成可恢復transaction。
4. 題目問原檔 logical block 2，即 block 10，與新追加的 11–13 無關。contiguous 直接讀 block 10，共1次 data read；embedded linked 從 head 8 讀出下一指標，再讀9、10，共3次 block reads；indexed 若 index cached 為1次 data read，cold index 為1次 index read 加1次 data read。

檢查。 位元約定、extent選擇、bitmap字串、inode與I/O成本須一致。

OS12-E06　Directory index 與 crash recovery 綜合

1. buckets：0{}、1{}、2{delta,epsilon,eta}、3{gamma}、4{beta,zeta}、5{}、6{alpha}、7{}。成功probe總和=1+1+1+1+2+2+3=11，除以7約1.571。
2. 本題是chaining，8 buckets可放超過8個names，不因第9個name必然滿；可因load factor選擇rehash。open addressing容量才受每slot一entry限制。並行rehash需鎖、版本或完整搬移協定，使reader不會漏掉只搬一半的entry。
3. commit durable前no-steal保證home仍舊，丟棄未提交log即回到old；commit durable後，完整after-images已持久，先redo所有metadata才提供mount服務，得到size24576與新pointers/bitmap；home已完成但log未清再redo仍同一結果。若after-image是size←24576，重放不會變36KiB；若用size+=12288則不是冪等redo。
4. 不保證。metadata journal保護namespace與allocation結構，不自動保證user data contents；本題使用ordered journal，data blocks11–13完成後才commit metadata。若只做metadata journal，recovery後仍可能讀到未持久data。

檢查。 hash統計、索引切換與每個crash prefix的最終可見狀態一致。

OS12-E07　多層索引與稀疏檔案的邊界

1. 5MiB+123=5243003。q=1280、d123；扣12direct與1024single後244=0×1024+244，故double[0][244]再加123。不可把1280直接除1024而忘了前面1036塊。
2. single為[12,1036)，即12–1035；double為[1036,1049612)，即1036–1049611。兩層index各一次I/O加data一次，共3次；第一層已cached則2次，所有indices cached則1次。
3. 題設讀取起點位於 file size 內的合法 hole，該 hole 中位於 EOF 之前的部分返回零位元組值而無須配置 data block；若請求跨 EOF，回傳長度至多到 EOF。若起點在 EOF 或之後則返回 0 bytes，兩者不同。file logical size可以很大而實際allocated bytes很小。仍可能需讀上層metadata判斷hole，但若上層pointer已為0，可停止，不必假造不存在的data I/O。

檢查。 容量、索引邊界、byte offset與實際metadata I/O分開計。

OS12-E08　open、dup與EOF的共同狀態

1. 初始P1:4→O1、P2:9→O2，(0,0)。read8192後(8192,0)。dup令P1:6→O1，仍(8192,0)。透過fd6讀4096後(12288,0)。P2 lseek後(12288,4096)。最後read在EOF回0，仍(12288,4096)。
2. 回0；O1.offset=size，沒有剩餘bytes。lseek只改O2而非inode內某個global offset，因此不改P1的位置。所有fd可引用同一inode，但不因此共享description。
3. 可以，close(fd6)只移除一個O1引用，fd4仍持有。fork繼承的fd4引用同一O1，故透過任一方read/lseek都改共用offset；它與兩次獨立open產生O1/O2不同。

檢查。 共享inode、共享description、共享fd table是不同關係。

OS12-E09　Redo不變量與非冪等反例

1. commit前no-steal使home=M0，丟未提交log仍M0。commit後log含完整M1 after-images，重放bitmap、pointers、size後均M1，即使home曾部分新也被覆寫。checkpoint已持久則home本來M1，log未清再redo仍M1；清log必須在checkpoint durable之後。
2. 兩次+=令size12288→24576→36864，append也重複11–13，破壞size/mapping。改成size←24576、pointers←[8,9,10,11,12,13]及bitmap←完整新值，兩次賦值等於一次，才是冪等after-image redo。
3. inode可能已指11–13而bitmap仍說free；丟log後半更新仍在home，其他檔可重複配置同塊。需禁止未提交home更新，或保留undo before-images並按協定復原，不能沿用no-steal證明。
4. 不夠。incremental B2只含週一後變更，必須B0→B1→B2並重放刪除/改名紀錄；differential B2才包含自B0後所有變更。snapshot與原volume共裝置，整台/整盤毀損可能一起遺失，不能替代獨立故障域backup。

檢查。 完整枚舉crash prefixes檢查實作；一般證明依commit、no-steal、冪等redo與持久順序。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 12.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study